

Chapter 1

Introduction

1.1 Introduction and Motivation

Indoor plants have been shown to improve mental well-being, reduce stress, and enhance indoor air quality [1]. However, plant care remains a challenge for many individuals, particularly those with busy professional or academic schedules. Irregular watering is one of the most common causes of houseplant mortality, often resulting from forgetfulness or extended periods of absence from home. Members of this project group had direct experience of this problem: one member lost a houseplant to root rot caused by overwatering during a period of inconsistent care, which motivated the group's interest in a system that could respond to actual soil conditions rather than relying on guesswork.

Traditional solutions to this problem include mechanical watering timers and passive capillary wicking systems. While these methods provide a degree of automation, they operate on fixed schedules or passive water transfer mechanisms, without any feedback from the actual soil moisture conditions. Consequently, they are unable to adapt to variations in plant species, pot size, ambient temperature, or seasonal changes in water requirements, often leading to overwatering or under-watering.

The rapid development of Internet of Things (IoT) technology has enabled low-cost, sensor-driven automation systems that can monitor environmental conditions in real time and respond accordingly [2]. By integrating soil moisture sensing with cloud-based communication, an IoT-enabled irrigation system can provide adaptive watering based on actual plant needs, while also offering remote monitoring

and control capabilities via a smartphone application.

This project presents the design and implementation of an IoT-based self-watering plant monitoring and irrigation system. The system utilises an ESP32 microcontroller to interface with soil moisture and water level sensors, a relay-controlled water pump, and the Blynk IoT platform for remote smartphone access. The following chapters describe the system design, implementation methodology, testing results, and a critical evaluation of the project outcomes.

1.2 Objectives

The following objectives were established for the project:

1. Design and implement an ESP32-based embedded system capable of continuous, autonomous soil moisture monitoring.
2. Develop an automatic irrigation mechanism that activates the water pump when soil moisture falls below a configurable threshold.
3. Integrate a water level detection subsystem to prevent pump dry-running and alert the user when the reservoir requires refilling.
4. Implement remote monitoring and manual watering control through a smartphone application using the Blynk IoT platform.
5. Establish reliable wireless connectivity between the ESP32 and the cloud server for real-time data communication.
6. Design a modular, well-structured, and maintainable embedded software architecture in C++.
7. Construct a functional physical prototype that demonstrates all system capabilities in a realistic usage scenario.

Chapter 2

Materials and Methods

2.1 System Overview

The self-watering plant system can be understood in terms of four functional layers, as illustrated in Figure 2.1. Sensors, a potentiometer, and a manual push button form the input layer, providing environmental readings and user settings to the ESP32 microcontroller, which serves as the central control unit. The ESP32 runs a state-machine-based algorithm that processes these inputs and drives the output devices: a relay-controlled water pump, a 16×2 character LCD for local status display, and an LED for low-water warnings. In parallel, the ESP32 maintains a Wi-Fi connection to the Blynk cloud platform, enabling remote monitoring and control from a smartphone application.

The input layer provides environmental data and user settings to the ESP32. The capacitive soil moisture sensor continuously measures soil moisture as a voltage, while the water level sensor monitors the reservoir status. A potentiometer allows the user to adjust the watering setpoint threshold locally. A physical push button allows manual watering. The ESP32 processes these inputs through a state-machine-based control algorithm. On the output side, a relay module switches the water pump, a 16×2 LCD displays system status, and an LED provides a visual low-water warning. Simultaneously, the ESP32 communicates with the Blynk cloud server over Wi-Fi.

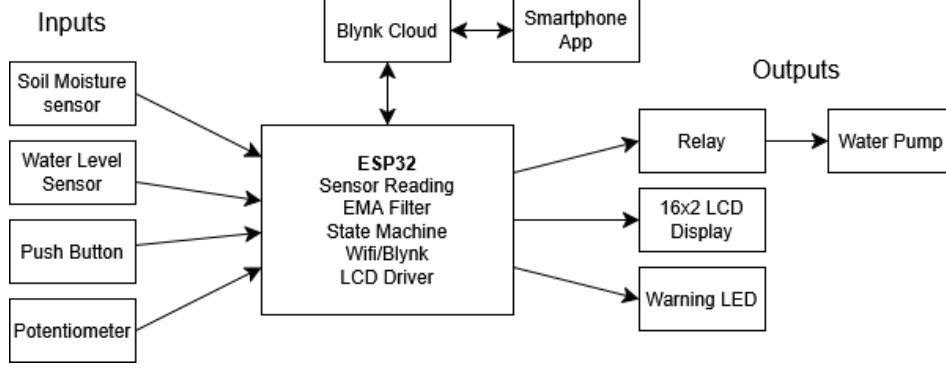


Figure 2.1: System block diagram of the IoT self-watering plant system

2.2 Materials

The hardware and software components employed in this project are summarised in Table 2.1. The ESP32 NodeMCU-32S was selected as the central microcontroller after the group evaluated several alternatives. An Arduino Uno was considered but lacks built-in Wi-Fi, which would have required an additional shield module and increased both cost and complexity. A Raspberry Pi was also discussed but was judged to be over-specified for the task, with higher power consumption and a higher price point than necessary for a single-sensor control loop. The ESP32 offered the best combination of integrated 2.4 GHz Wi-Fi, multiple analogue-to-digital converter (ADC) channels, sufficient GPIO pins, low cost (under £5), and straightforward programming via the Arduino framework [3].

A capacitive soil moisture sensor was chosen over a resistive alternative, as capacitive sensors do not suffer from electrode corrosion when in prolonged contact with moist soil, resulting in improved long-term reliability. The sensor output is a voltage proportional to the dielectric permittivity of the surrounding soil, which varies with moisture content.

A 16×2 character LCD with an I2C backpack was integrated to provide local status feedback without requiring a network connection. The display shows the current moisture percentage, the user-defined setpoint, tank status, and pump state. A 10 kΩ potentiometer allows the user to physically adjust the moisture set-point threshold in real time, providing immediate control without the smartphone application.

The water pump is a mini submersible type, driven via a single-channel relay

module. An external 9 V battery provides independent power to the pump circuit, isolating it from the ESP32’s power supply to prevent voltage drops and potential microcontroller resets during pump activation.

Table 2.1: Hardware components used in the system

Component	Model / Spec	Purpose	Qty
Microcontroller	ESP32 NodeMCU-32S	Central processing and Wi-Fi communication	1
Soil Moisture Sensor	Capacitive v1.2	Measure soil moisture level (analogue)	1
Water Level Sensor	Resistive float	Detect low water level in reservoir	1
Relay Module	5 V single-ch.	Switch pump power circuit	1
Water Pump	Mini submersible	Pump water from reservoir to plant	1
LCD Display	16×2 I2C	Display moisture, setpoint, tank and pump status	1
Potentiometer	10 k Ω	Manually adjust moisture set-point threshold	1
LED	5 mm Red	Low water warning indicator	1
Push Button	Tactile switch	Manual watering trigger	1
Power Supply	9 V battery	Independent pump power	1
Breadboard	Full-size	Prototyping connections	1
Jumper Wires	M-to-M, M-to-F	Circuit connections	–
Water Reservoir	Plastic container	Water storage	1
Silicone Tubing	6 mm I.D.	Water delivery to plant pot	1

The software was developed in PlatformIO IDE [4] using the Espressif32 platform and the Arduino framework. The Blynk IoT library and the LiquidCrystal_I2C library were included as dependencies. All embedded software was written in C++ following a modular file structure.

2.3 Circuit Design

The ESP32 is unable to directly drive the water pump, as its GPIO pins provide a maximum output current of approximately 12 mA at 3.3 V [3], which is insufficient for the pump motor. Therefore, a 5 V single-channel relay module was employed as an intermediary power switch. The ESP32 controls the relay’s input pin (GPIO25) using a digital HIGH/LOW signal, and the relay in turn switches the pump’s external 9 V power supply. This configuration electrically isolates the high-current

pump circuit from the low-power microcontroller circuit.

All low-power components, including the soil moisture sensor, water level sensor, potentiometer, LED, push button, and LCD module, are powered directly from the ESP32 supply rails, as their current requirements are within the microcontroller's operating limits. These components share a common ground with the ESP32 and do not require separate power supplies.

Electrical isolation is only required for the water pump circuit, which is achieved using the relay module. This ensures that the high-current pump circuit does not interfere with the low-power control circuitry.

The complete GPIO pin assignment is presented in Table 2.2. The soil moisture sensor is connected to GPIO34 (ADC1 channel 6), while the potentiometer for threshold adjustment is connected to GPIO35 (ADC1 channel 7). Both are input-only pins dedicated to analogue reading. GPIO27 reads the digital output of the water level sensor, where a HIGH reading indicates low water. GPIO14 drives the warning LED, and GPIO13 reads the manual push button input using the internal pull-up resistor (active LOW when pressed). The 16×2 LCD communicates via the I2C bus, using GPIO21 (SDA) and GPIO22 (SCL), with an I2C address of 0x3F.

Table 2.2: ESP32 GPIO pin assignments

GPIO Pin	Connected To	Function
GPIO34 (ADC1.CH6)	Soil Moisture Sensor	Analogue moisture voltage reading
GPIO35 (ADC1.CH7)	Potentiometer	Analogue setpoint threshold input
GPIO27	Water Level Sensor	Digital water level detection (HIGH = low)
GPIO25	Relay Module (IN)	Pump on/off control
GPIO14	LED (via resistor)	Low water warning indicator
GPIO13	Push Button (to GND)	Manual watering trigger (active LOW)
GPIO21 (SDA)	LCD I2C Data	I2C data line for LCD display
GPIO22 (SCL)	LCD I2C Clock	I2C clock line for LCD display

The circuit schematic is shown in Figure 2.2.

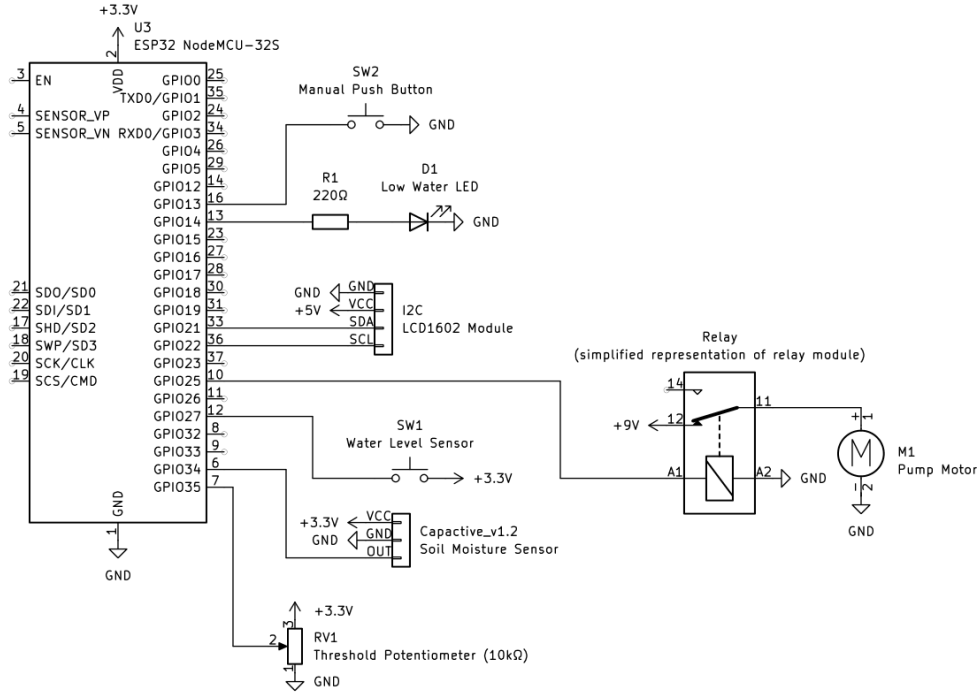


Figure 2.2: Circuit schematic diagram of the self-watering plant system

2.4 Software Design

2.4.1 Software Architecture and Module Design

A modular software architecture was adopted to improve code readability, facilitate independent testing of each subsystem, and simplify debugging. The firmware was organised into seven header/source file pairs, each encapsulating a specific hardware interface or control function. The module structure is illustrated in Figure 2.3 and summarised below:

- **config.h** — Central configuration file defining all GPIO pin numbers, Blynk credentials, Wi-Fi credentials, calibration constants, watering timing parameters, and LCD settings.
- **soil_sensor.h/.cpp** — Encapsulates the capacitive soil moisture sensor. Provides `readVoltageAvg()` which reads the ADC using `analogReadMilliVolts()` and returns the result in volts, with optional multi-sample averaging.

- `relay.h/.cpp` — Encapsulates the relay module with configurable active-HIGH or active-LOW logic. Provides `on()`, `off()`, and `isOn()` methods.
- `water_level.h/.cpp` — Reads the digital water level sensor with configurable polarity. Provides `isLow()` and `isOk()` boolean accessors.
- `indicator_led.h/.cpp` — Controls the warning LED with configurable active-HIGH/LOW polarity.
- `watering_controller.h/.cpp` — Implements the core irrigation state machine with four states (`IDLE`, `WATERING`, `SOAK`, `BLOCKED`), dual-threshold logic, and manual watering support.
- `main.cpp` — The top-level entry point containing `setup()` and `loop()`, Blynk initialisation, EMA sensor filtering, button debouncing, potentiometer reading, LCD updating, and the main execution loop.

This modular design allowed individual components to be tested independently before system integration, significantly reducing debugging time.

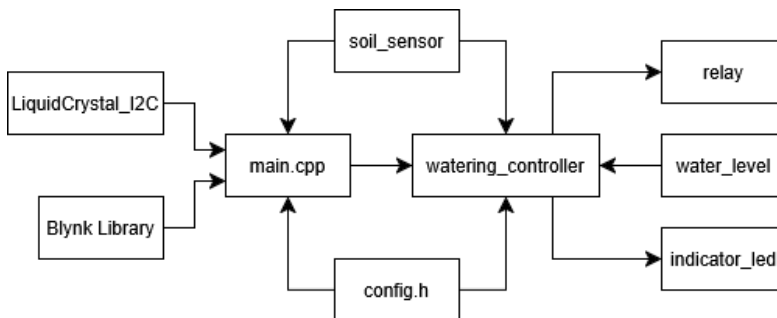


Figure 2.3: Software module dependency diagram

2.4.2 Core Watering Logic

The automatic watering algorithm is implemented as a finite state machine within the `WateringController` class, operating through four distinct states: `IDLE`, `WATERING`, `SOAK`, and `BLOCKED`. The decision logic is illustrated in Figure 2.4.

Sensor Reading and EMA Filtering. The soil moisture sensor returns an analogue voltage via the ESP32’s ADC, converted to volts using `analogReadMilliVolts()`. To suppress transient noise and avoid false watering triggers, an Exponential Moving Average (EMA) filter is applied in the main loop:

$$V_{\text{flt}}(n) = 0.9 \cdot V_{\text{flt}}(n - 1) + 0.1 \cdot V_{\text{raw}}(n) \quad (2.1)$$

Each new reading contributes only 10% to the filtered value. The smoothing factor $\alpha = 0.1$ was chosen based on the relationship between the filter’s time constant and the physical dynamics of soil moisture. For an EMA filter, the equivalent time constant τ can be approximated as [5]:

$$\tau \approx \frac{1}{\alpha} = \frac{1}{0.1} = 10 \text{ samples} \quad (2.2)$$

With the main loop executing at approximately 500ms intervals, this yields a time constant of roughly 5 seconds. Soil moisture is an inherently slow-changing physical quantity — even during active watering, water requires several seconds to percolate through soil and reach the buried sensor. A 5-second time constant therefore provides sufficient smoothing to reject transient ADC noise and brief contact disturbances (for example, the sensor being nudged during physical adjustments), while remaining responsive enough to track genuine moisture changes within a single watering-soak cycle (10s total). During early development, a larger smoothing factor ($\alpha = 0.5$) was briefly tested but resulted in the controller occasionally triggering a watering pulse in response to a momentary ADC spike when the sensor was touched; reducing α to 0.1 eliminated this issue. Conversely, setting α to a much smaller value (e.g. 0.02) would delay the controller’s reaction to completed watering by over 20 seconds, risking unnecessary additional watering cycles before the filter settled [6].

Voltage-to-Percentage Conversion. The filtered voltage is converted to a moisture percentage using the calibration constants `SOIL_V_DRY` (0.80 V $\approx$ 0%) and `SOIL_V_WET` (2.00 V $\approx$ 100%). The conversion is linear between these two endpoints.

Dual-Threshold Watering Logic. The dual-threshold approach was not the group’s initial design. Early development used a simple single-threshold comparison: if the moisture reading fell below the setpoint, the pump would activate, and if it rose above, the pump would stop. During bench testing, this produced noticeable oscillation — the pump would switch on, deliver a brief pulse of water, the moisture reading would momentarily rise above the threshold, the pump would stop, and within seconds the reading would fall back below as the water had not yet percolated through the soil, triggering another pulse. This rapid on/off cycling was both inefficient and potentially damaging to the relay contacts.

Through group discussion, two modifications were proposed and implemented. First, a hysteresis margin of 3% was added to the setpoint threshold, so that the

pump would only stop when moisture exceeded the setpoint *plus* hysteresis, preventing re-triggering from small fluctuations near the boundary. Second, the concept was extended to use two voltage thresholds derived from the user-adjustable setpoint percentage, allowing the controller to distinguish between moderate and severe dryness:

- **Threshold X** — the primary watering trigger. When $V_{\text{filt}} < X$, the soil is considered dry and a *small* watering pulse (1000 ms) is initiated.
- **Threshold Y** ($Y < X$) — the severe dryness trigger. When $V_{\text{filt}} < Y$, a *large* watering pulse (1000 ms) is initiated.

Note: the controller distinguishes between moderate and severe dryness using two thresholds; in the final implementation, due to the bench plant pot size, both trigger a 1000 ms watering pulse while preserving the extensible dual-threshold control structure.

The 3% hysteresis margin was determined experimentally: values below 2% were insufficient to prevent occasional re-triggering, while values above 5% caused the system to tolerate noticeably dry soil before stopping, which felt unresponsive during demonstration. The setpoint is adjusted locally via the potentiometer (GPIO35) and mirrored to the Blynk application on Virtual Pin V10 for remote monitoring.

State Machine. The controller cycles through four states:

1. **IDLE** — Monitors the filtered soil voltage. If $V < X$, transitions to **WATERING**.
2. **WATERING** — Relay is energised, driving the pump. After the pulse duration expires, relay is de-energised and the system transitions to **SOAK**.
3. **SOAK** — Waits 9000 ms for water to percolate through soil before re-reading the sensor. Returns to **IDLE**.
4. **BLOCKED** — Entered when water level sensor detects low reservoir. All watering is inhibited. LED flashes at 1 Hz. Returns to **IDLE** when reservoir is refilled.

Manual Watering Override. A manual watering request can be triggered by pressing the physical push button (GPIO13, with software debouncing at 30 ms) or via the Blynk application (Virtual Pin V11). Both initiate a single small watering pulse if the water level is adequate.

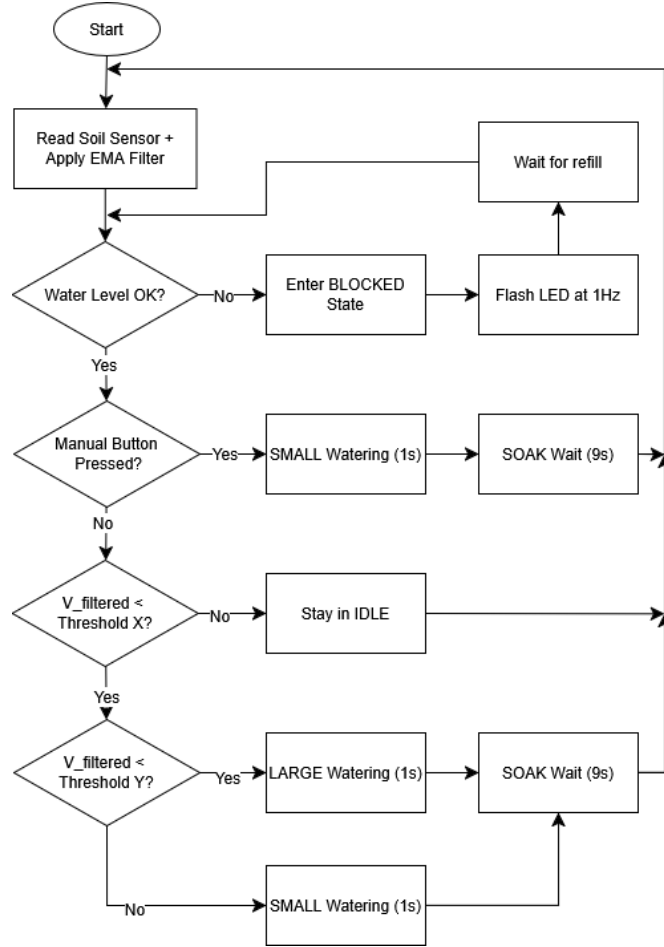


Figure 2.4: Flowchart of the automatic watering state machine

2.4.3 IoT Remote Control via Blynk

The Blynk IoT platform was selected for remote connectivity due to its native support for ESP32, ease of integration, and provision of a customisable mobile application interface without requiring mobile app development [7]. The ESP32 connects to the Blynk cloud server over Wi-Fi, maintaining a persistent connection through the `Blynk.run()` function called within the main loop.

Five Blynk virtual pins were configured to provide comprehensive remote monitoring and control, as summarised in Table 2.3.

Table 2.3: Blynk virtual pin configuration

Pin	Direction	Function
V0	ESP32 → App	Soil moisture percentage (gauge widget)
V1	ESP32 → App	Water tank status (1 = OK, 0 = LOW)
V2	ESP32 → App	Pump state (1 = ON, 0 = OFF)
V10	ESP32 → App	Current moisture setpoint percentage
V11	App → ESP32	Manual watering trigger button (momentary)

The ESP32 pushes updated sensor data to V0, V1, V2, and V10 at one-second intervals using a `BlynkTimer` callback. When the user presses the manual watering button (V11), the ESP32 receives the command through a `BLYNK_WRITE(V11)` callback. If the water tank is adequate, a small watering pulse is triggered; otherwise, the request is rejected. The button is immediately reset to zero to maintain momentary behaviour.

The Blynk application interface is presented in Figure 2.5.

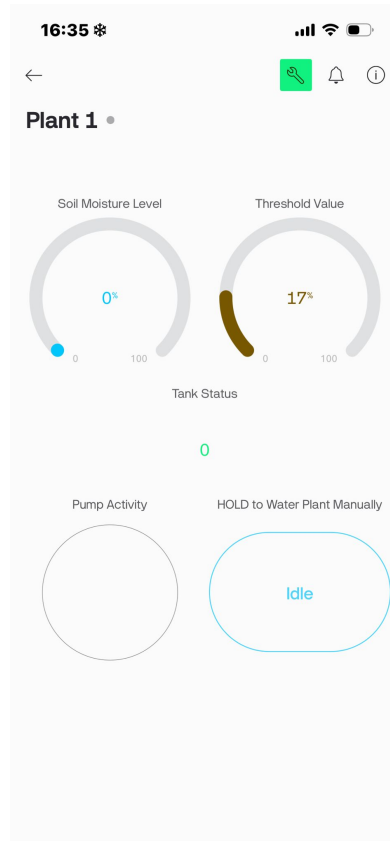


Figure 2.5: Blynk mobile application interface

2.4.4 Network Configuration and Connectivity Challenges

A significant technical challenge encountered during development was establishing reliable Wi-Fi connectivity for the ESP32. The ESP32 hardware only supports the 2.4 GHz Wi-Fi band (802.11 b/g/n) and cannot connect to 5 GHz networks [3]. This limitation presented three distinct connectivity obstacles during the project.

Attempt 1: Mobile Phone Hotspot. The initial approach was to create a Wi-Fi hotspot using a team member’s smartphone. However, modern smartphones typically default to 5 GHz hotspot broadcasting, and on some devices the frequency band cannot be manually configured. Consequently, the ESP32 was unable to detect or connect to the hotspot.

Attempt 2: MacBook Internet Sharing. An alternative approach was explored using macOS Internet Sharing to create a software-based Wi-Fi access point. While this feature can theoretically create a 2.4 GHz network, the connection proved unreliable during testing, with frequent disconnections.

Attempt 3: University Campus Wi-Fi. The university’s eduroam Wi-Fi network was also considered. However, enterprise Wi-Fi networks employ WPA2-Enterprise authentication with a captive portal login page, which the ESP32 cannot handle as it lacks a web browser interface.

Successful Solution: Portable Router Relay. The problem was ultimately resolved by introducing a portable wireless router. The mobile phone creates a 4G/5G data hotspot, the router connects to it as a client and rebroadcasts as a dedicated 2.4 GHz access point. The ESP32 connects to this 2.4 GHz network successfully. This topology is illustrated in Figure 2.6.

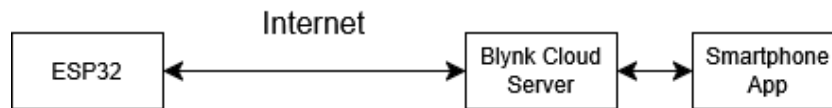


Figure 2.6: Network topology: router relay solution for ESP32 Wi-Fi connectivity

This solution demonstrated the importance of understanding hardware-level constraints and developing creative workarounds within a limited development environment.

2.5 Physical Assembly and Construction

The physical prototype was assembled to replicate a realistic indoor plant watering scenario. The system consists of three main physical units: the plant pot assembly, the water reservoir, and the electronic control enclosure.

The soil moisture sensor was inserted vertically into the plant pot soil, positioned near the root zone. A length of silicone tubing, perforated with small holes, was arranged in a loop around the base of the pot interior to distribute water evenly.

The water reservoir consists of a plastic container positioned adjacent to the plant pot. The submersible water pump and the water level sensor are placed inside this reservoir. The pump outlet connects to the perforated tubing via a silicone tube.

The ESP32, relay module, LCD display, potentiometer, and associated wiring were housed in a separate enclosure. The push button and warning LED are mounted on the exterior for easy user access. The annotated photograph of the complete system is shown in Figure 2.7.

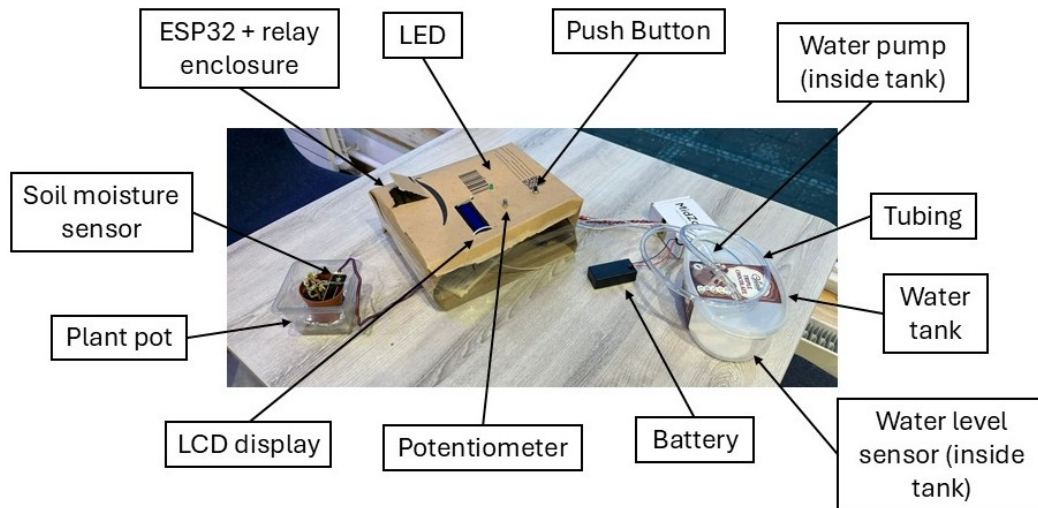


Figure 2.7: Annotated photograph of the fully assembled self-watering plant system

Chapter 3

Results and Analysis

This chapter presents the results of component-level testing and integrated system evaluation. A bottom-up testing strategy was adopted: each hardware subsystem (sensor, relay, water level switch) was first verified independently using serial monitor output and visual inspection, before the complete system was assembled and tested as an integrated unit. This approach allowed faults to be isolated at the module level, which proved particularly valuable after the ESP32 hardware failure discussed in Section 4.1.2. The IoT remote control functionality was tested last, as it depended on all local subsystems operating correctly.

3.1 Sensor Calibration and Testing

The capacitive soil moisture sensor was calibrated by recording the ESP32 ADC output (converted to voltage using `analogReadMilliVolts()`) under four controlled conditions. The calibration results are presented in Table 3.1.

Table 3.1: Soil moisture sensor calibration data

Test Condition	Voltage (V)	Mapped Percentage
Sensor in open air	0.70	<0% (below dry cal.)
Sensor in dry soil	0.85	4%
Sensor in moist soil	1.50	58%
Sensor submerged in water	2.10	>100% (above wet cal.)

*All voltage and percentage values are approximate experimental readings.

The calibration results confirm a consistent and monotonic relationship between sensor voltage and soil moisture conditions. Although the response is not perfectly linear, the linear approximation is sufficient for threshold-based control. The system does not require high absolute accuracy, as control decisions are based on relative changes in moisture level.

The selected calibration points (0.80 V for dry and 2.00 V for wet) provided stable and repeatable readings during testing. The chosen threshold of 45% (approximately 1.34 V) resulted in reliable triggering of the watering cycle without false activation. This demonstrates that the calibration method is appropriate for practical operation.

Based on these calibration results, the dry-point voltage was set to 0.80 V (0% moisture) and the wet-point voltage to 2.00 V (100% moisture) in the firmware configuration. The default watering setpoint was configured at 45%, mapping to approximately 1.34 V. The ESP32's `analogReadMilliVolts()` function applies Espressif's internal calibration lookup table, partially mitigating the known ADC non-linearity [8].

```

src > C:\main\cpp > C:\main\cpp
void setup() {
  48 Serial.begin(115200);
  49 delay(200);
  50
  51 // Set ADC resolution to 12-bit (0-4095)
  52 // Note: analogReadMilliVolts handles calibration internally
  53 analogReadResolution(12);
  54
  55 Serial.println("---- Soil Moisture Sensor ADC Test Started ----");
  56 Serial.println("Tip: Test in AIR for Dry Point, then in WATER for Wet Point.");
  57 }
  58
  59 void loop() {
  60 // Get the smoothed average voltage
  61 float V = readVoltageAvg();
  62
  63 // Calculate the estimated moisture percentage
  64 float M = voltageToMoisturePct(V);
  65
  66 // Format and print data to the Serial Monitor
  67 // %3f = 3 decimal places, %1f = 1 decimal place
  68 Serial.printf("Pin=%d | Voltage=%3f V | Est. Moisture=%1f %\n", ADC_PIN, V, M);
  69
  70 // Update data twice per second
  71 delay(500);
  72 }
  73
  74
  
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

Executing task: platformio device monitor

```

Pin=34 | Voltage=0.142 V | Est. Moisture=1.9 %
Pin=34 | Voltage=0.142 V | Est. Moisture=1.9 %
Pin=34 | Voltage=0.142 V | Est. Moisture=1.9 %
Pin=34 | Voltage=0.142 V | Est. Moisture=1.9 %
Pin=34 | Voltage=0.142 V | Est. Moisture=1.9 %
Pin=34 | Voltage=1.833 V | Est. Moisture=78.8 %
=====Moisture=93.9 %
Pin=34 | Voltage=2.183 V | Est. Moisture=94.7 %
Pin=34 | Voltage=2.132 V | Est. Moisture=92.4 %
Pin=34 | Voltage=2.084 V | Est. Moisture=91.1 %
Pin=34 | Voltage=2.073 V | Est. Moisture=89.7 %
Pin=34 | Voltage=2.054 V | Est. Moisture=88.8 %
=====Moisture=88.8 %
Pin=34 | Voltage=1.879 V | Est. Moisture=88.9 %
Pin=34 | Voltage=0.142 V | Est. Moisture=1.9 %
Pin=34 | Voltage=0.142 V | Est. Moisture=1.9 %
Pin=34 | Voltage=0.142 V | Est. Moisture=1.9 %
Pin=34 | Voltage=0.142 V | Est. Moisture=1.9 %
Pin=34 | Voltage=0.142 V | Est. Moisture=1.9 %
  
```

Figure 3.1: PlatformIO serial monitor output during sensor calibration: the sensor was tested in air (~ 0.14 V) and then submerged in water (~ 2.18 V), confirming the expected voltage range

The calibration behaviour was verified using the PlatformIO serial monitor during bench testing, as shown in Figure 3.1. When the sensor was held in open air,

the serial output consistently reported a voltage of approximately 0.142 V with an estimated moisture of 1.9%. Upon submerging the sensor tip in water, the voltage rose sharply to approximately 2.18 V (estimated moisture 94.7%), with intermediate readings visible during the transition. This live output confirmed the monotonic voltage–moisture relationship observed in Table 3.1 and demonstrated that the EMA filter tracked the genuine step change within a few seconds while suppressing the brief transient fluctuations visible during the air-to-water transition.

3.2 Pump and Relay Testing

The relay module was tested by repeatedly toggling GPIO25 between HIGH and LOW while monitoring the relay’s onboard indicator LED and listening for the mechanical click of the switching contact. Over approximately 30 manual switching cycles, the relay engaged and disengaged without any missed triggers or spurious activations. The pump, connected via the relay’s normally-open terminal to the 9 V battery, started and stopped cleanly on each cycle. No perceptible delay was observed between the GPIO transition and the pump response, consistent with the relay’s specified mechanical response time of approximately 10 ms.

During these tests, the ESP32’s serial output and LCD display continued to operate without interruption or reset, confirming that the external 9 V pump circuit was effectively isolated from the 3.3 V control logic. This was a particularly important verification given the earlier hardware failure described in Section 4.1.2, which had demonstrated the consequences of inadequate power domain separation.

3.3 Automatic Watering System Test

The automatic watering system was evaluated by inserting the sensor into a pot of dry soil and observing the controller’s response over several consecutive watering cycles. The setpoint was configured to 45% (approximately 1.34 V) via the potentiometer. Table 3.2 summarises the approximate readings observed during a representative test sequence.

Table 3.2: Observed watering cycle data during automatic operation (setpoint = 45%)

Cycle	Pre-water Voltage (V)	Pre-water Moisture (%)	Post-soak Voltage (V)	Post-soak Moisture (%)	Action
1	1.18	32	1.30	42	Re-water
2	1.30	42	1.41	51	Stop
3*	1.36	47	—	—	Idle

*Re-checked approximately 10 minutes after Cycle 2. All values are approximate.

In Cycle 1, the filtered voltage was approximately 1.18 V (32% moisture), well below the 45% setpoint. The controller triggered a 1 s watering pulse, followed by the 9 s soak delay. After soaking, the voltage had risen to approximately 1.30 V (42%), which was still below the setpoint (accounting for the 3% hysteresis margin), so a second cycle was initiated. After Cycle 2, the post-soak reading reached approximately 1.41 V (51%), exceeding the setpoint plus hysteresis, and the controller returned to the IDLE state. When the sensor was re-checked approximately ten minutes later, the moisture had settled to around 47% as excess water drained through the soil, remaining above the setpoint and confirming that no unnecessary additional watering was triggered.

The EMA filtering contributed noticeably to the stability of this process. During the soak period, the raw ADC readings fluctuated by approximately ± 0.03 V as water percolated unevenly past the sensor; the filtered output, however, changed smoothly and monotonically, preventing the controller from misinterpreting a brief dip as a dry condition. No oscillatory on/off behaviour was observed at any point during testing.

3.4 IoT Remote Control Test

Upon successful Wi-Fi connection via the router relay configuration, the ESP32 established communication with the Blynk cloud server. The soil moisture gauge on the mobile application displayed updated readings, confirming successful data transmission. The remote watering button was pressed, and the water pump activated within approximately 1–2 seconds, demonstrating acceptable response latency.

The system maintained stable communication with the Blynk cloud, with no disconnections observed during testing. Sensor data updates were received at regular intervals, providing near real-time feedback to the user. The measured latency of 1–2 seconds between issuing a command and pump activation is acceptable for a domestic IoT application and does not impact usability.

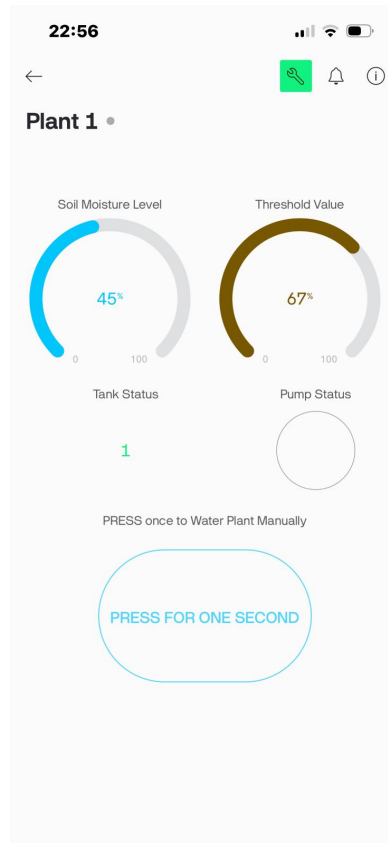


Figure 3.2: Blynk application displaying real-time data during operation

As shown in Figure 3.2, the Blynk mobile application provides real-time monitoring of the system. The interface displays the current soil moisture level, user-defined threshold, tank status, and pump activity. This allows the user to quickly assess system conditions and intervene if necessary. The integration of both monitoring and control functions within a single interface enhances usability and supports remote operation.

3.5 Water Level Safety Test

The water level safety mechanism was validated by simulating a low-water condition using the float switch. When the sensor indicated low water (logic HIGH), the controller immediately entered the BLOCKED state, disabling both automatic and manual watering functions. The warning LED was activated, providing clear visual feedback of the fault condition.

Upon restoring the water level (logic LOW), the system exited the BLOCKED state and resumed normal operation without requiring a reset. This confirms that the safety mechanism operates reliably and prevents dry running of the pump, which is critical for system longevity and safe operation.

3.6 Error Analysis

Despite successful system operation, several sources of error and uncertainty were identified that may affect measurement accuracy and system performance. Where possible, approximate magnitudes are provided based on observations during testing and manufacturer specifications.

ADC Non-linearity. The ESP32's internal ADC is known to exhibit non-linear behaviour, particularly near 0 V and above 2.8 V [8]. The `analogReadMilliVolts()` function applies Espressif's factory calibration lookup table, which improves accuracy in the mid-range but does not fully eliminate the non-linearity. According to the ESP-IDF documentation, the corrected ADC output has a typical accuracy of approximately $\pm 10\text{--}20\text{ mV}$ in the 0.15–2.45 V range [8]. Given the calibration span of 1.20 V (from 0.80 V to 2.00 V), this corresponds to a moisture reading uncertainty of approximately $\pm 1\text{--}2\%$. During bench testing, the serial monitor output showed that repeated readings of a stationary sensor in dry soil varied by no more than $\pm 5\text{ mV}$ over a 30-second window, suggesting that short-term ADC repeatability is good and that the dominant ADC error contribution is systematic offset rather than random noise.

Sensor Precision and Positioning. The capacitive sensor measures the dielectric permittivity of the material immediately surrounding it, which is sensitive to sensor insertion depth and orientation. During testing, repositioning the sensor approximately 2 cm deeper into the same pot produced a voltage change

of roughly 0.05–0.08 V, corresponding to a shift of approximately 4–7% in the reported moisture. This is a significant source of variability and means that meaningful comparisons between different pots or soil types would require recalibration. For a single-pot deployment where the sensor remains fixed, this error is incurred only once and does not affect the system’s ability to track relative changes over time.

Environmental Factors. Temperature variations affect both the dielectric constant of water in soil and the capacitance of the sensor element itself. No temperature compensation was implemented in this system. Testing was conducted indoors at a stable room temperature (approximately 20–22 °C), so temperature-induced drift was not observed during the project. However, in an outdoor or greenhouse deployment where temperature may vary by 20 °C or more over a day, this could introduce measurable drift that the current design does not account for.

Network Latency. Remote control commands issued via the Blynk mobile application were observed to take approximately 1–2 seconds to result in pump activation. This delay includes the round-trip time from the smartphone to the Blynk cloud server and then to the ESP32 over Wi-Fi. For a domestic plant watering application, this latency is negligible — a one-second delay in watering onset has no practical impact on plant health. However, the latency could become relevant in applications requiring tighter real-time control.

The combined effect of these error sources means that the displayed moisture percentage should be regarded as an approximate, application-specific indicator rather than an absolute measurement. Nevertheless, the system’s threshold-based control approach is inherently tolerant of moderate measurement offsets, since the controller responds to whether the reading is above or below a user-adjustable setpoint, not to the absolute value itself.

Chapter 4

Discussion and Conclusions

4.1 Discussion of Results

The experimental results presented in Chapter 3 confirm that the system meets its core functional requirements: soil moisture is monitored continuously, watering is triggered automatically when moisture falls below a user-defined threshold, the pump is protected against dry-running, and real-time data is accessible remotely via the Blynk smartphone application.

4.1.1 Comparison with Commercial Products

Commercially available smart plant monitors such as the Xiaomi Flora Monitor [9] and the Parrot Pot [10] offer similar sensing capabilities, typically including soil moisture, temperature, and ambient light measurement, with companion smartphone applications that provide historical data logging and plant-specific care recommendations. The system developed in this project offers comparable core irrigation functionality at a total component cost of approximately £15–20 compared to retail prices of £40–80 for commercial products. It additionally provides features not commonly found in consumer products, including a local LCD readout, a physical potentiometer for threshold adjustment without requiring a phone, and a manual push-button override. However, the commercial products benefit from injection-moulded enclosures, rechargeable batteries, multi-sensor capability, and extensive cloud-based data history — features that were beyond the scope of a four-week student project.

4.1.2 Hardware Failure and Engineering Lessons

During the prototyping phase, one ESP32 board was permanently damaged when the USB cable (connected to a laptop for serial debugging) and the external 9 V battery powering the pump circuit were both connected simultaneously. The most likely cause was a ground-loop current or reverse voltage path through the relay module’s common ground, which allowed the higher battery voltage to feed back into the ESP32’s power rails. The ESP32 datasheet specifies an absolute maximum input voltage of 3.6 V on any GPIO pin [3]; exceeding this would cause irreversible damage to the internal ADC or I/O circuitry.

This incident highlighted a significant gap in the original circuit design: no reverse-polarity protection or supply isolation mechanism was included. In a revised design, a Schottky diode (e.g. 1N5819) could be placed in series with the battery supply line to prevent reverse current flow into the USB-powered portion of the circuit. Alternatively, a dedicated voltage regulator with reverse-current blocking could isolate the two power domains entirely. As an immediate corrective measure for the remainder of the project, the team adopted a strict protocol of disconnecting the battery before plugging in the USB cable for programming or debugging, which prevented further incidents but added inconvenience to the development workflow.

This experience reinforced the importance of considering failure modes during the circuit design stage, rather than relying solely on careful manual procedures during development. In a commercial or long-term deployment, such reliance on user discipline would be unacceptable, and hardware-level protection would be essential.

4.1.3 Project Time Constraints and Testing Gaps

The project was completed within a four-week timeframe, which was sufficient to deliver a working prototype but imposed significant limitations on the depth of testing and validation that could be performed. The system was developed and tested using a single medium-sized indoor plant pot with standard potting compost. No systematic testing was conducted with different plant species, pot sizes, or soil types, all of which would substantially affect the required watering volume, drainage rate, and optimal setpoint value.

For example, a small succulent in a shallow pot would likely require a much lower setpoint (perhaps 20–25%) and a shorter pump pulse than the 45% threshold and 1-second pulse used in testing, whereas a large fern in a deep pot might need a higher setpoint and multiple longer pulses to achieve adequate moisture penetration. The current fixed 1-second pulse duration was determined empirically to work well for the specific bench test pot, but it is not necessarily transferable to other configurations. Ideally, a series of controlled experiments would have been conducted across at least three to four different pot sizes and soil types to characterise the relationship between pulse duration, soil volume, and moisture rise, producing a lookup table or empirical formula that could inform the firmware defaults. The four-week schedule did not permit this level of characterisation.

Similarly, long-term reliability testing — running the system continuously for days or weeks to observe battery depletion, sensor drift, and Wi-Fi stability — was not feasible within the project timeline. The results presented in Chapter 3 represent short-duration bench tests (typically 10–30 minutes per test session), which are sufficient to verify functional correctness but do not fully characterise the system’s behaviour under sustained real-world operation.

4.1.4 Critical Review of Design Decisions

Several design choices made early in the project, while pragmatically justified at the time, would benefit from reconsideration in a future iteration.

Blynk vs. MQTT. The Blynk platform was selected primarily for its ease of integration: the Arduino library provides ready-made functions for connecting to the cloud and binding virtual pins to widgets, which allowed the team to achieve a working IoT demonstration within a few days. However, the free-tier Blynk account imposes limitations on the number of datastreams and does not support historical data logging. An alternative approach using the MQTT protocol with an open-source broker (e.g. Mosquitto) and a self-hosted dashboard (e.g. Node-RED or Grafana) would have offered greater flexibility, full data history, and no dependency on a third-party commercial platform. The trade-off would have been significantly more development time for the server-side infrastructure, which was judged to be impractical within the four-week timeframe.

Breadboard vs. PCB. The entire circuit was assembled on a solderless breadboard, which enabled rapid prototyping and easy modification during development.

However, breadboard connections are inherently unreliable: during transport of the prototype between the laboratory and the demonstration venue, several jumper wires became dislodged and required re-seating before the system would operate correctly. A custom PCB, or at minimum a stripboard with soldered connections, would have provided significantly better mechanical reliability. For a four-week project, the breadboard was an acceptable compromise, but it would not be suitable for any deployment beyond a controlled bench demonstration.

Wi-Fi connectivity. The ESP32's limitation to the 2.4 GHz band created an unexpected obstacle, as described in Section 2.4.4. The university campus network uses WPA2-Enterprise authentication with a captive portal, which the ESP32 cannot negotiate without a web browser interface. The eventual solution — using a portable travel router to rebroadcast a phone hotspot on 2.4 GHz — worked reliably but added cost and complexity. In hindsight, this constraint should have been identified during the initial research phase rather than discovered during integration testing, which would have allowed the team to budget time for the workaround earlier in the project. For a home deployment, most domestic routers broadcast on 2.4 GHz by default, so this issue would typically not arise.

4.1.5 Calibration Validity

Although the system performed reliably in demonstration tests, the dependence on calibration constants and environmental conditions means that the moisture percentage should be interpreted as an application-specific relative estimate rather than an absolute agronomic measurement. The calibration points (0.80 V for dry and 2.00 V for wet) were determined empirically for one particular sensor unit in one particular soil type. A different sensor of the same model, or the same sensor in a different soil, could produce different voltage ranges, requiring recalibration of the firmware constants. This is a known limitation of low-cost capacitive sensors. However, it does not significantly affect the system since it relies on threshold-based control rather than absolute accuracy.

4.1.6 Team Workflow and Crisis Recovery

The three group members divided responsibilities along functional lines: one member took primary responsibility for the embedded C++ software and control al-

gorithm, one focused on the physical circuit design, soldering, and wiring, and one concentrated on the Blynk application interface, smartphone integration, and project management. All three contributed to report writing. This division allowed parallel progress during the first three weeks, with each member able to develop and unit-test their subsystem independently before integration.

The most significant disruption to this workflow occurred in Week 4 when the soldered PCB was destroyed by the accidental 9 V supply incident described in Section 4.1.2. At that point, the team had to make a rapid decision under time pressure: attempt to solder a replacement PCB, or revert to the breadboard configuration that had been used successfully during early development. After a brief discussion, the group chose the breadboard approach, reasoning that it had already been proven functional and could be reconstructed within a day, whereas soldering a new PCB carried the risk of repeating the same failure if the root cause had not been fully identified. This decision reduced mechanical reliability slightly (as discussed in Section 4.1.4) but allowed the system to be rebuilt quickly and tested in time. During that final week, the team met four times outside of scheduled sessions to rebuild the circuit, re-verify all subsystems, and complete the remaining software features (LCD display and potentiometer threshold control) that had not yet been integrated.

In retrospect, the PCB failure, while stressful at the time, led to a simpler and more debuggable final design. The breadboard made it straightforward to probe signals, swap components, and adjust wiring during the final testing phase. A key lesson for future projects was that maintaining a known-good fallback configuration throughout development — rather than committing entirely to a more advanced build — provides valuable resilience against unexpected hardware failures.

4.2 Evaluation Against Objectives

Table 4.1 presents an evaluation of the project outcomes against the stated objectives.

Table 4.1: Evaluation of project outcomes against stated objectives

Objective	Status	Evidence
Autonomous soil moisture monitoring	Met	Section 3.1
Automatic threshold-based irrigation	Met	Section 3.3
Water level detection and pump protection	Met	Section 3.5
Remote monitoring and control via Blynk	Met	Section 3.4
Reliable wireless connectivity	Met	Section 2.4.4
Modular C++ software architecture	Met	Section 2.4.1
Functional physical prototype	Met	Section 2.5

All seven project objectives were successfully met. It should be noted, however, that “met” indicates functional demonstration under bench test conditions; as discussed above, the system has not been validated under the full range of real-world conditions that a deployed product would encounter.

4.3 Limitations

The current system supports only a single plant with a single soil moisture sensor. Extending the design to monitor multiple pots would require either additional analogue input channels (the ESP32 has several unused ADC1 pins) or an analogue multiplexer, together with modifications to the software state machine to manage multiple independent watering schedules. This was not attempted within the four-week project timeframe.

The Blynk free-tier account used in this project does not retain historical data beyond the current session. As a result, there is no way for the user to review past moisture trends, watering events, or system faults after the fact. A future iteration could address this by logging data to an SD card locally or by upgrading to a Blynk plan with data history support.

Reliable Wi-Fi connectivity required the use of a portable router to bridge the 2.4 GHz gap, as discussed in Section 2.4.4. This adds cost (approximately £10–15 for a basic travel router) and an extra point of failure. In a home environment with an existing 2.4 GHz network, the router would be unnecessary, but the current design cannot be guaranteed to work in all Wi-Fi environments without this workaround.

The 9 V alkaline battery powering the pump has a typical capacity of approximately 500 mAh. Given that the pump draws roughly 100–150 mA during each 1-second watering pulse, and assuming an average of 5–10 watering cycles per day, the battery would be expected to last in the order of several weeks before requiring replacement. No battery voltage monitoring was implemented, so the user receives no advance warning of impending battery depletion.

Finally, the dual-threshold watering algorithm uses fixed pulse durations and a single user-adjustable setpoint. It does not adapt automatically to different plant species, pot sizes, or soil types, all of which affect how much water is needed and how quickly it drains. Tuning the system for a different plant currently requires manual recalibration of the setpoint and potentially the timing constants.

4.4 Applications

The system has potential applications in domestic plant care, commercial plant maintenance, greenhouse and urban farming, and integration with smart home ecosystems such as Amazon Alexa or Google Home.

4.5 Ethical, Environmental and Sustainability Considerations

Environmental Impact. The sensor-driven approach promotes water conservation by delivering water only when needed, eliminating waste from fixed-schedule over-watering.

Ethical Considerations. The Blynk IoT platform transmits sensor data to a third-party cloud server, raising data privacy considerations. Although limited to moisture readings, users should be informed about data handling practices.

Sustainability. Electronic components contribute to e-waste at end-of-life. The 9 V disposable battery is a particular concern; future iterations could use rechargeable batteries or solar charging.

Regulatory Compliance. The ESP32 module is designed for operation in the 2.4 GHz ISM band and is documented by the manufacturer for use in standards-

compliant products [3]. Any commercial product based on this design would require full regulatory assessment at system level.

4.6 Commercialisation and Intellectual Property

The total bill of materials is estimated at approximately £15–20. Commercial smart plant products retail for £40–80, suggesting a viable margin for a productised version with improved packaging and a custom PCB. The intellectual property resides primarily in the software implementation and system integration design.

4.7 Future Work

Several directions for future development have been identified, informed by the limitations and design reflections discussed above.

The most immediately impactful improvement would be **multi-plant support**. The ESP32 has several unused ADC1 channels, and the modular software architecture was designed with extensibility in mind — adding a second `SoilSensor` and `WateringController` instance would require relatively modest code changes, though a second relay and pump channel would also be needed on the hardware side. A more scalable approach for larger numbers of plants would use an analogue multiplexer (e.g. CD4051) to share a single ADC channel across multiple sensors.

Data logging and historical visualisation would significantly enhance the system’s utility. Implementing local logging to a micro-SD card module, or migrating the IoT backend from Blynk to an MQTT broker with a Grafana dashboard, would allow users to review moisture trends over days or weeks and identify patterns such as seasonal changes in water consumption.

An **adaptive watering algorithm** could replace the current fixed-pulse approach. By recording the moisture rise produced by each watering pulse, the controller could learn the relationship between pulse duration and moisture change for a given pot and soil combination, automatically adjusting the pulse length over successive cycles. This would address the current limitation of requiring manual tuning when changing plant or pot type.

Solar-powered operation would eliminate the dependency on disposable 9 V batteries. A small solar panel with a lithium-polymer battery and charge controller could provide sustainable autonomous power, particularly for outdoor or balcony deployments where sunlight is available.

Finally, **OTA (Over-The-Air) firmware updates** via Wi-Fi would allow software improvements and bug fixes to be deployed without physical access to the device, which would be essential for any commercially deployed version of the system.

4.8 Conclusions

This project has successfully designed, implemented, and tested an IoT-based self-watering plant monitoring and irrigation system within a four-week development timeframe. All seven stated project objectives were achieved and demonstrated under bench test conditions. The system monitors soil moisture using an EMA-filtered capacitive sensor, automatically initiates watering through a dual-threshold state machine with hysteresis, prevents pump dry-running through water level detection, and provides remote smartphone monitoring and control via the Blynk platform.

The project encountered and resolved several significant engineering challenges. The ESP32's restriction to 2.4 GHz Wi-Fi, combined with the university campus network's WPA2-Enterprise authentication, required the introduction of a portable router relay — a solution that, while functional, highlighted the importance of identifying hardware constraints early in the design process. The destruction of an ESP32 board due to simultaneous USB and battery connection exposed a gap in the circuit's power protection design and led to a better understanding of supply isolation requirements.

The main strengths of the project lie in the modular C++ software architecture, which facilitated independent testing and debugging; the dual-threshold state machine with hysteresis, which provided stable closed-loop control without oscillation; and the successful integration of local feedback (LCD and LED) with cloud-based remote monitoring at low cost. The primary limitations are the restriction to a single plant and sensor, the absence of historical data logging, the reliance on an external router for Wi-Fi in certain environments, and the lack of

automated adaptation to different plant species or pot configurations. Future work could address these through multi-sensor expansion, migration to an MQTT-based IoT backend, adaptive watering algorithms, and solar-powered operation.

References

- [1] K. T. Han and L. W. Ruan, “Effects of Indoor Plants on Self-Reported Perceptions,” <https://doi.org/10.3390/su11164506>, 2019, accessed: 2025-03-01.
- [2] N. Gondchawar and R. S. Kawitkar, “IoT Based Smart Agriculture,” *IJAR-CCE*, vol. 5, no. 6, pp. 838–842, 2016.
- [3] Espressif Systems, “ESP32 Series Datasheet,” https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf, 2024, accessed: 2025-03-01.
- [4] PlatformIO Labs, “PlatformIO: Professional Collaborative Platform for Embedded Development,” <https://platformio.org/>, 2024, accessed: 2025-03-01.
- [5] S. W. Smith, *The Scientist and Engineer’s Guide to Digital Signal Processing*, 1st ed. California Technical Publishing, 1997, chapter 19: Recursive Filters.
- [6] J. G. Proakis and D. G. Manolakis, *Digital Signal Processing: Principles, Algorithms, and Applications*, 4th ed. Pearson, 2007, chapter 9: Filter Design Techniques.
- [7] Blynk Inc., “Blynk IoT Platform Documentation,” <https://docs.blynk.io/en/>, 2024, accessed: 2025-03-01.
- [8] Espressif Systems, “ADC Calibration – ESP-IDF Programming Guide,” https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/peripherals/adc_calibration.html, 2023, accessed: 2025-03-01.
- [9] Xiaomi, “Flower care smart plant monitor,” <https://www.amazon.de/dp/B01MUDQD8I>, 2023, also known as Mi Flora; accessed: 2025-03-01.
- [10] Parrot, “Parrot pot – smart, connected flower pot,” <https://www.amazon.com/Parrot-Pot-Smart-Connected-Flower/dp/B01KV0JCI4>, 2016, product discontinued; accessed: 2025-03-01.